

CSC 321

Introduction to Digital Design and Microprocessors (3 Credits)

Lecture Note 1

Number System and Arithmetic

Arithmetic operations is a branch of mathematics, that involves the study of numbers, operation of numbers that are useful in all the other [branches of mathematics](#). It basically comprises operations such as Addition, Subtraction, Multiplication and Division.

These basic mathematical operations (+, -, ×, and ÷) we use in our everyday life. Whether we need to calculate the annual budget or distribute something equally to a number of people, for every such aspect of our life, we use arithmetic operations.

Let us understand each of the basic arithmetic operations in detail.

Basic Arithmetic Operations

The four basic arithmetic operations in Maths, for all real numbers, are:

- Addition (Finding the Sum; '+')
- Subtraction (Finding the difference; '-')
- Multiplication (Finding the product; '×')
- Division (Finding the quotient; '÷')

Addition

The addition is a mathematical process of adding things together. The addition process is denoted by '+' sign. It involves combining two or more numbers into a single term. In addition process, the order does not matter. It means that the addition process is commutative. It can involve any type of number whether it be a real or complex number, fraction, or decimals.

Example: $4.13 + 3.87 = 8$

The addition of more than two numbers, values or terms is also known as a summation of terms and can involve n number of values.

Addition Rules

The following are the addition rules for integers:

- Addition of two positive integers is a positive integer
- Addition of two negative integers is a negative integer
- While adding positive and negative integers, subtract the integers and use the sign of the largest integer number

Subtraction

The subtraction operation gives the difference between two numbers. Subtraction is denoted by '-' sign. It is almost similar to addition but is the conjugate of the second term. It is the

inverse process of addition. The addition of the term with the negative term is known as subtraction. This process is mostly used to find how many are left when some things are taken away.

Example: $15 - 7$

The term can also be re-written as $15 + (-7)$

Adding terms we have, 8.

Subtraction Rules

The following are the subtraction rules for integers:

- If both the signs of the integers are positive, the answer will be the positive integer
- If both the signs of the integers are negative, the answer will be the negative integer
- If the signs of the integers are different, subtract the values, and take the sign from the largest integer value.

Multiplication

Multiplication is known as repeated addition. It is denoted by ' $\times$ ' or '*'. It also combines with two or more values to result in a single value. The multiplication process involves multiplicand, multiplier. The result of the multiplication of multiplicand and the multiplier is called the product

Example: $2 \times 3 = 6$

Here, "2" is the multiplier, "3" is the multiplicand, and the result "6" is called the product.

The product of two numbers says 'a' and 'b' results in a single value term 'ab,' where a and b are the factors of the final value obtained.

Multiplication Rules

The following are the multiplication rules for the integers.

- The product of two positive integers is a positive integer
- The product of two negative integers is a positive integer
- The product of positive and negative integer is a negative integer

Division

The division is usually denoted by ' $\div$ ' and is the inverse of multiplication. It constitutes two terms dividend and divisor, where the dividend is divided by the divisor to give a single term value. When the dividend is greater than the divisor, the result obtained is greater than 1, or else it would be less than 1.

Example: $4 \div 2 = 2$

Here, "4" is the dividend, "2" is the divisor, and the result "2" is called the quotient.

Division Rules

The following are the division rules for integers:

- The division of two positive integers is a positive integer
- The division of two negative numbers is a positive integer
- The division of integers with different signs results in the negative integer.

Basic Arithmetic Properties

The basic arithmetic properties for real numbers are:

- Commutative property
- Associative property
- Distributive property

Commutative Property

This property is applicable only for two arithmetic operations, i.e., addition and multiplication.

Suppose A and B are two numbers, then, according to commutative property;

$$A+B = B+A \quad \text{Example: } 1 + 2 = 2 + 1$$

$$A \times B = B \times A \quad \text{Example: } 1 \times 2 = 2 \times 1$$

Thus, the order of numbers in addition and multiplication does not change the result.

Associative Property

Like commutative property, the associative property is also applicable to addition and multiplication.

$$A+(B+C) = (A+B)+C \quad \text{Example: } 1 + (2+3) = (1+2)+3$$

$$A \times (B \times C) = (A \times B) \times C \quad \text{Example: } 1 \times (2 \times 3) = (1 \times 2) \times 3$$

Thus, if we change the grouping of numbers, the result does not change.

Distributive Property

According to the distributive property, if A, B and C are any three real numbers, then,

$$A \times (B + C) = A \times B + A \times C$$

$$\text{Example: } 2 \times (3 + 4) = (2 \times 3) + (2 \times 4)$$

$$2 \times 7 = 6 + 8$$

$$14 = 14$$

Hence, proved.

NUMBER SYSTEMS

The number system involves using symbols representing numbers on a line with rules. These symbols, called digits, range from 0 to 9.

The number system provides a structured way to express numbers using digits, ensuring clarity and consistency in mathematical calculations. It facilitates various arithmetic operations like addition, subtraction, multiplication, and division.

Number systems are systems in mathematics that are used to express numbers in various forms and are understood by computers. A number is a mathematical value used for counting and measuring objects, and for performing arithmetic calculations. A number system is a system representing numbers. It is also called the system of numeration and it defines a set of values to represent a quantity. These numbers are used as digits and the most common ones are 0 and 1, that are used to represent binary numbers. Digits from 0 to 9 are used to represent other types of number systems.

A number system is defined as the representation of numbers by using digits or other symbols in a consistent manner. The value of any digit in a number can be determined by a digit, its position in the number, and the base of the number system. The numbers are represented in a unique manner and allow us to operate arithmetic operations like addition, subtraction, and division.

Numbers have various categories like natural numbers, whole numbers, rational and irrational numbers, and so on. Similarly, there are various types of number systems that have different properties, like the binary number system, the octal number system, the decimal number system, and the hexadecimal number system.

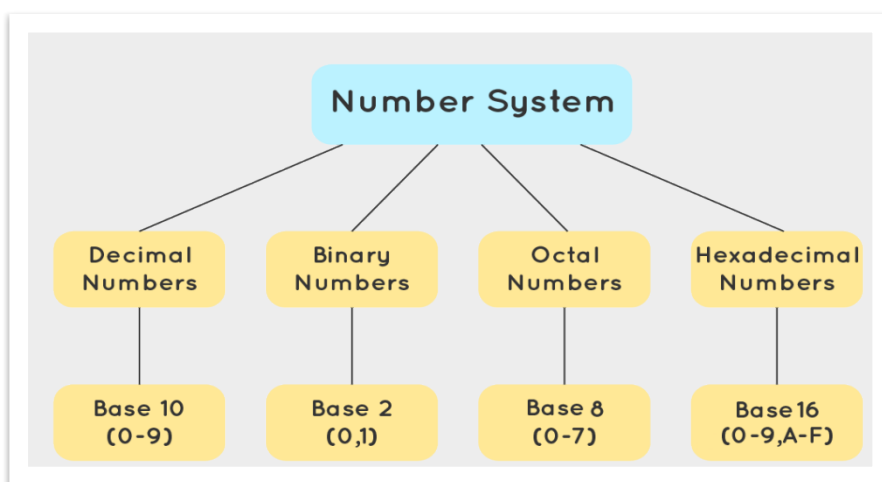
Types of Number Systems

There are different types of number systems in which the four main types are as follows.

- Binary number system (Base - 2)
- Octal number system (Base - 8)
- Decimal number system (Base - 10)
- Hexadecimal number system (Base - 16)

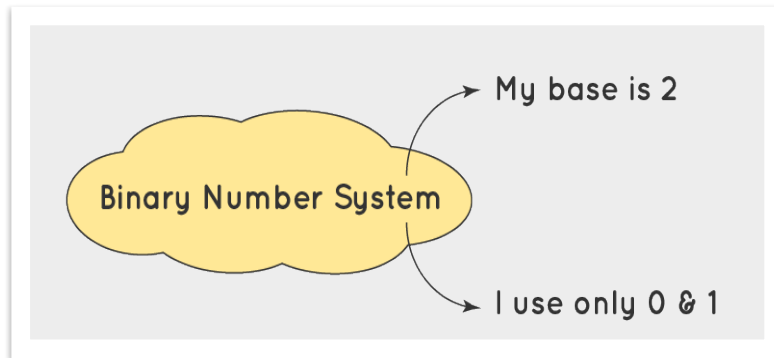
Number System Chart

Given below is a chart of the main four types of number system that we use to represent numbers.



Binary Number System

The binary number system uses only two digits: 0 and 1. The numbers in this system have a base of 2. Digits 0 and 1 are called bits and 8 bits together make a byte. The data in computers is stored in terms of bits and bytes. The binary number system does not deal with other numbers such as 2,3,4,5 and so on. For example: 10001_2 , 111101_2 , 1010101_2 are some examples of numbers in the binary number system.



Arithmetic Operations of Binary Numbers:

In the binary number system, numbers are shown using only two symbols: 0 and 1. This method serves as the foundation for all current computer systems, where each symbol's worth in a binary number is determined by multiples of 2.

When we look at the binary number 1011_2 , it actually equals the decimal number 11. In the binary system, each digit holds a place value based on powers of 2. The rightmost digit signifies 2^0 , the next one to the left signifies 2^1 , and this pattern continues with increasing powers of 2 for each subsequent digit.

In the Binary System, there are only two symbols or possible digit values, i.e., 0 and 1. Represented by any device that only 2 operating states or possible conditions. Generally, it is expressed by subscript 2 or binary (B).

Binary arithmetic is essential part of various digital systems. You can add, subtract, multiply, and divide binary numbers using various methods. These operations are much easier than decimal number arithmetic operations because binary system has only two digits: 0 and 1.

Binary additions and subtractions are performed as same in decimal additions and subtractions. When we perform binary additions, there will have two outputs: Sum (S) and Carry (C) . There are four rules for binary addition. These are given as following below,

Input A	Input B	Sum (S) A+B	Carry (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

There are four rules for binary subtraction. These are given as following below,

Input A	Input B	Subtract (S) A-B	Borrow (B)
0	0	0	0
0	1	0	1
1	0	1	0
1	1	0	0

Borrow 1 is required from next higher order bit to subtract 1 from 0. So, result became 0.

There are four rules for binary multiplication. These are given as following below,

Input A	Input B	Multiply (M) AxB
0	0	0
0	1	0
1	0	0
1	1	1

Whenever at least one input is 0, then multiplication is always 0.

There are four part in any division: Dividend, Divisor, quotient, and remainder. These are given as following rules for binary division,

Input A	Input B	Divide (D) A/B
0	0	Not defined
0	1	0
1	0	Not defined
1	1	1

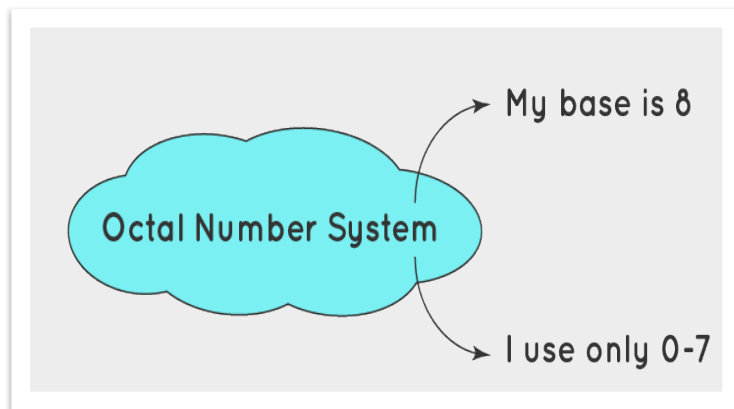
Whenever divisor is 0, then result is always not defined.

Octal Number System

The octal number system utilizes eight digits from 0 to 7. While less commonly used today, it finds applications in computing, particularly when dealing with machine code and permissions. Octal numbers are often used as shorthand representations of binary values.

An example of the octal number system is the number 53. In octal, this number is represented as 65. Each digit in the octal system ranges from 0 to 7, making it a base-8 numbering system.

The octal number system uses eight digits: 0,1,2,3,4,5,6 and 7 with the base of 8. The advantage of this system is that it has lesser digits when compared to several other systems, hence, there would be fewer computational errors. Digits like 8 and 9 are not included in the octal number system. Just like the binary, the octal number system is used in minicomputers but with digits from 0 to 7. For example, 35_8 , 23_8 , and 141_8 are some examples of numbers in the octal number system.



Arithmetic Operations of Octal Numbers:

It has base of 8 digits ranging from 0 to 7 (i.e., 0, 1, 2, 3, 4, 5, 6, 7). Generally, it is expressed by subscript 8 or octal (o).

Arithmetic operations of octal numbers are also performed same as decimal arithmetic operations. The addition table for octal numbers is given as below:

+	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	1	2	3	4	5	6	7	10
2	2	3	4	5	6	7	10	11
3	3	4	5	6	7	10	11	12
4	4	5	6	7	10	11	12	13
5	5	6	7	10	11	12	13	14
6	6	7	10	11	12	13	14	15
7	7	10	11	12	13	14	15	16

With the help of above table, we can perform additions of octal numbers.

Similarly, we can perform subtraction of octal numbers same as decimal number subtractions. But you can only take borrow of a number 7, which is maximum digit allowed in octal number systems.

In the multiplication of octal numbers, if the product is less than radix of octal (i.e., 8), then we take it as the result, else divide it by radix of octal (i.e., 8) and take the remainder as the LSB (least significant bit). The quotient is taken as carry in the next significant digit.

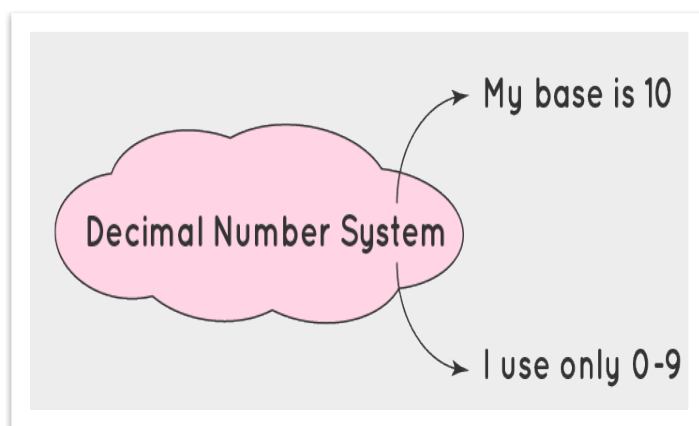
Similarly, division of octal numbers can be performed by following the rules of division of decimal numbers but maximum allowed digit will be 7.

Decimal Number System

The decimal number system is the most common and widely used number system in math. Its base value is 10, and it uses ten digits from 0 to 9. Each digit's value depends on its position within a number, with the rightmost digit representing ones, the next tens, and so forth.

In the math number system known as decimal, each digit has ten possible values from 0 to 9. For example, the number 4567 in the decimal system consists of the digits 4, 5, 6, and 7, with each digit representing a different power of 10. The value of each digit is determined by its position in the number.

The decimal number system uses ten digits: 0,1,2,3,4,5,6,7,8 and 9 with the base number as 10. The decimal number system is the system that we generally use to represent numbers in real life. If any number is represented without a base, it means that its base is 10. For example, 723_{10} , 32_{10} , and 4257_{10} are some examples of numbers in the decimal number system.



Arithmetic Operations of Decimal Numbers:

Arithmetic operations of decimal numbers are very popular and much easier. These operations are also performed as same in other number systems.

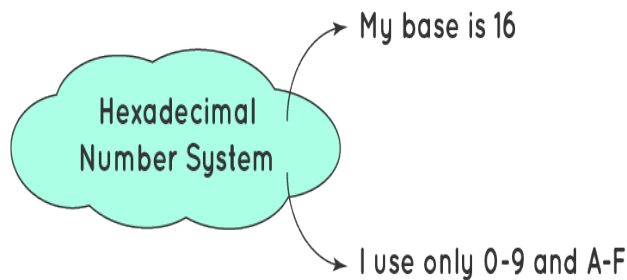
Hexadecimal Number System

The hexadecimal number system uses a base value of 16, with representations made using 16 symbols: 0 to 9, and A to F. Each symbol corresponds to a decimal value.

In hexadecimal, the number 255 is shown as FF. This is because F represents 15 in decimal, so two Fs together represent $15 \times 16 + 15 = 255$ in decimal.

The hexadecimal number system uses sixteen digits/alphabets: 0,1,2,3,4,5,6,7,8,9 and A,B,C,D,E,F with the base number as 16. Here, A-F of the hexadecimal system means the numbers 10-15 of the

decimal number system respectively. This system is used in computers to reduce the large-sized strings of the binary system. For example, $7B3_{16}$, $6F_{16}$, and $4B2A_{16}$ are some examples of numbers in the hexadecimal number system.



It has base of 16 digits ranging from 0 to F (i.e., 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, and A, B, C, D, E, F). A, B, C, D, E, F are equivalent single digits of 10, 11, 12, 13, 14, 15 respectively. Generally, it is expressed by subscript 16 or Hexa (H) or (h).

Arithmetic operations of hexadecimal numbers can be performed using addition table for hexadecimal numbers which is given as below:

+	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	10
2	2	3	4	5	6	7	8	9	A	B	C	D	E	F	10	11
3	3	4	5	6	7	8	9	A	B	C	D	E	F	10	11	12
4	4	5	6	7	8	9	A	B	C	D	E	F	10	11	12	13
5	5	6	7	8	9	A	B	C	D	E	F	10	11	12	13	14
6	6	7	8	9	A	B	C	D	E	F	10	11	12	13	14	15
7	7	8	9	A	B	C	D	E	F	10	11	12	13	14	15	16
8	8	9	A	B	C	D	E	F	10	11	12	13	14	15	16	17
9	9	A	B	C	D	E	F	10	11	12	13	14	15	16	17	18
A	A	B	C	D	E	F	10	11	12	13	14	15	16	17	18	19
B	B	C	D	E	F	10	11	12	13	14	15	16	17	18	19	1A
C	C	D	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B
D	D	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C
E	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D
F	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E

F	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E
---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

With the help of above table, we can perform additions of hexadecimal numbers.

Subtraction of hexadecimal numbers can be performed by using complement methods or simple as decimal subtractions. The rule of simple hexadecimal subtraction is the digit borrowed from the immediate higher place is counted as 16.

In the multiplication of hexadecimal numbers, if the product is less than radix of hexadecimal (i.e, 16), then we take it as the result, else divide it by radix of hexadecimal (i.e., 16) and take the remainder as the LSB (least significant bit). The quotient is taken as carry in the next significant digit. Using these rules, you can make a table for hexadecimal multiplications.

Similarly, division of hexadecimal numbers can be performed by following the rules of division of decimal numbers but maximum allowed digit will be F(=15 in decimal).

Conversion of Number Systems

A number can be converted from one number system to another number system using number system formulas. Like binary numbers can be converted to octal numbers and vice versa, octal numbers can be converted to decimal numbers and vice versa, and so on. Let us see the steps required in converting number systems.

Steps for Conversion of Binary to Decimal Number System

To convert a number from the binary to the decimal system, we use the following steps.

- **Step 1:** Multiply each digit of the given number, starting from the rightmost digit, with the exponents of the base.
- **Step 2:** The exponents should start with 0 and increase by 1 every time we move from right to left.
- **Step 3:** Simplify each of the above products and add them.

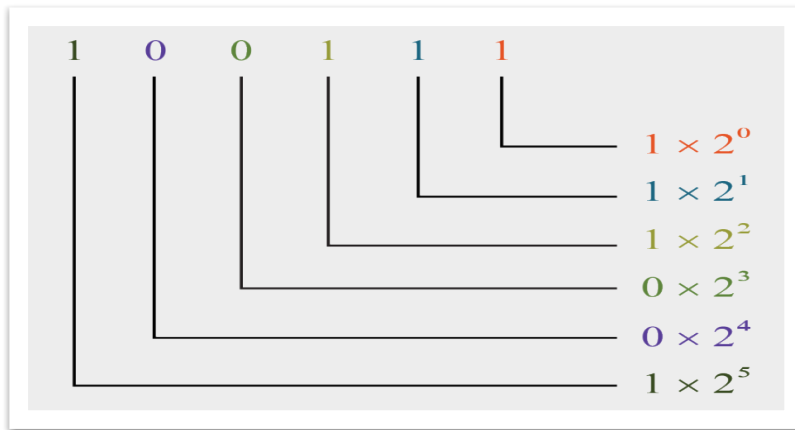
Let us understand the steps with the help of the following example in which we need to convert a number from binary to decimal number system.

Example: Convert 100111_2 into the decimal system.

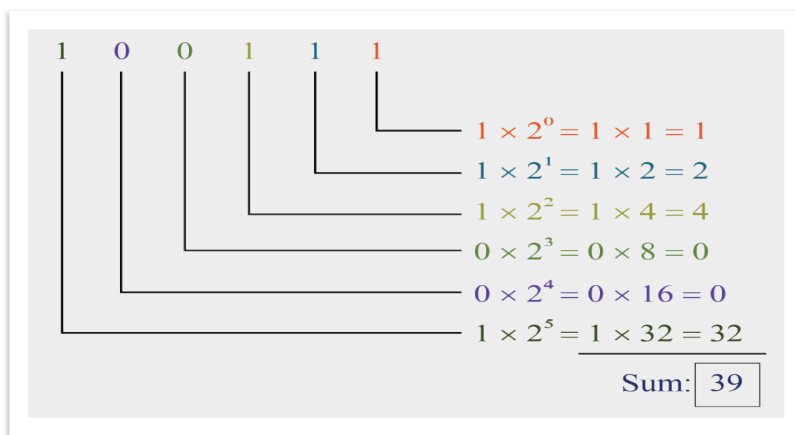
Solution:

Step 1: Identify the base of the given number. Here, the base of 100111_2 is 2.

Step 2: Multiply each digit of the given number, starting from the rightmost digit, with the [exponents](#) of the base. The exponents should start with 0 and increase by 1 every time as we move from right to left. Since the base is 2 here, we multiply the digits of the given number by 2^0 , 2^1 , 2^2 , and so on from right to left.



Step 3: We just simplify each of the above products and add them.



Here, the sum is the equivalent number in the decimal number system of the given number. Or, we can use the following steps to make this process simplified.

$$\begin{aligned}
 100111 &= (1 \times 2^5) + (0 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) \\
 &= (1 \times 32) + (0 \times 16) + (0 \times 8) + (1 \times 4) + (1 \times 2) + (1 \times 1) \\
 &= 32 + 0 + 0 + 4 + 2 + 1 \\
 &= 39
 \end{aligned}$$

Thus, $100111_2 = 39_{10}$.

Conversion of Decimal Number System to Binary / Octal / Hexadecimal Number System

To convert a number from the decimal number system to a binary/octal/hexadecimal number system, we use the following steps. The steps are shown on how to convert a number from the decimal system to the octal system.

Example: Convert 4320_{10} into the octal system.

Solution:

Step 1: Identify the base of the required number. Since we have to convert the given number into the octal system, the base of the required number is 8.

Step 2: Divide the given number by the base of the required number and note down the quotient and the remainder in the quotient-remainder form. Repeat this process (dividing the quotient again by the base) until we get the quotient less than the base.

8		4	3	2	0
8		5	4	0	— 0
8		6	7	— 4	
8		8	— 3		
		1	— 0		

Step 3: The given number in the octal number system is obtained just by reading all the remainders and the last quotient from bottom to top.

8		4	3	2	0
8		5	4	0	— 0
8		6	7	— 4	
8		8	— 3		
		1	— 0		

Therefore, $4320_{10} = 10340_8$

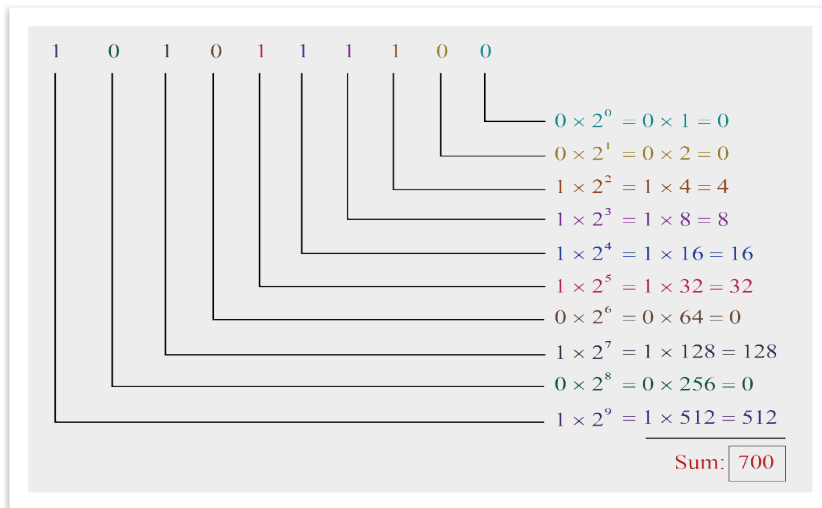
Conversion from One Number System to Another Number System

To convert a number from one of the binary/octal/hexadecimal systems to one of the other systems, we first convert it into the decimal system, and then we convert it to the required systems by using the above-mentioned processes.

Example: Convert 1010111100_2 to the hexadecimal system.

Solution:

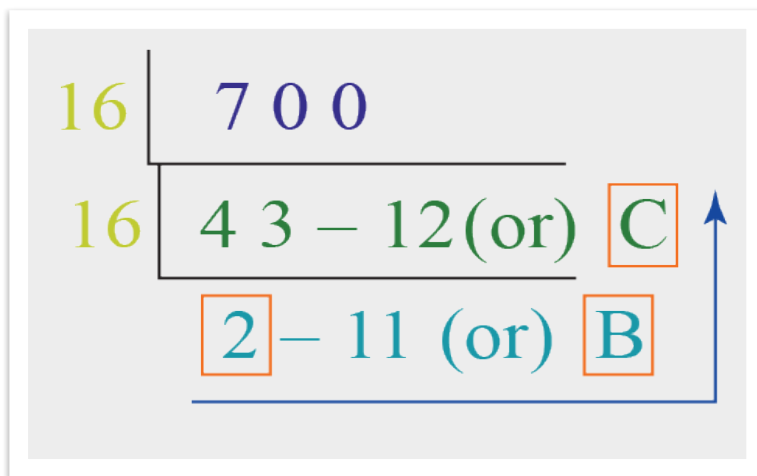
Step 1: Convert this number to the decimal number system as explained in the above process.



Thus, $1010111100_2 = 700_{10} \rightarrow (1)$

Step 2: Convert the above number (which is in the decimal system), into the required number system (hexadecimal).

Here, we have to convert 700_{10} into the hexadecimal system using the above-mentioned process. It should be noted that in the hexadecimal system, the numbers 11 and 12 are written as B and C respectively.



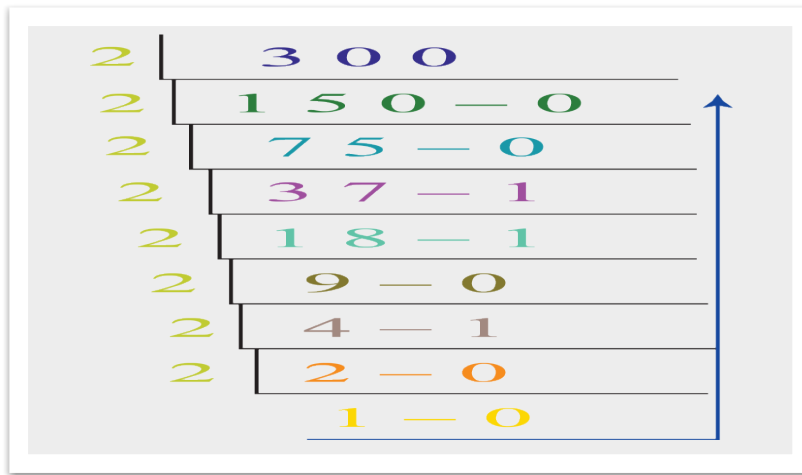
Thus, $700_{10} = 2BC_{16} \rightarrow (2)$

From the equations (1) and (2), $1010111100_2 = 2BC_{16}$

Number Systems Examples

1. **Example 1:** Convert 300_{10} into the binary number system with base 2.

Solution: 300_{10} is in the decimal system. We divide 300 by 2 and note down the quotient and the remainder. We will repeat this process for every quotient until we get a quotient that is less than 2.

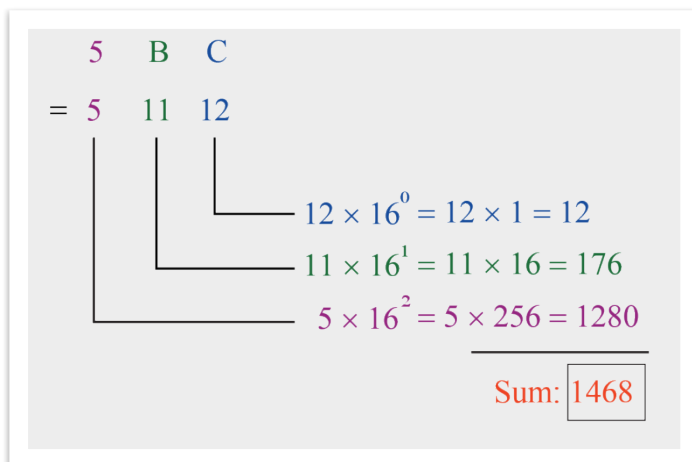


The equivalent number in the binary system is obtained by reading all the remainders and just the last quotient from bottom to top as shown above.

Thus, $300_{10} = 100101100_2$

2. **Example 2:** Convert $5BC_{16}$ into the decimal system.

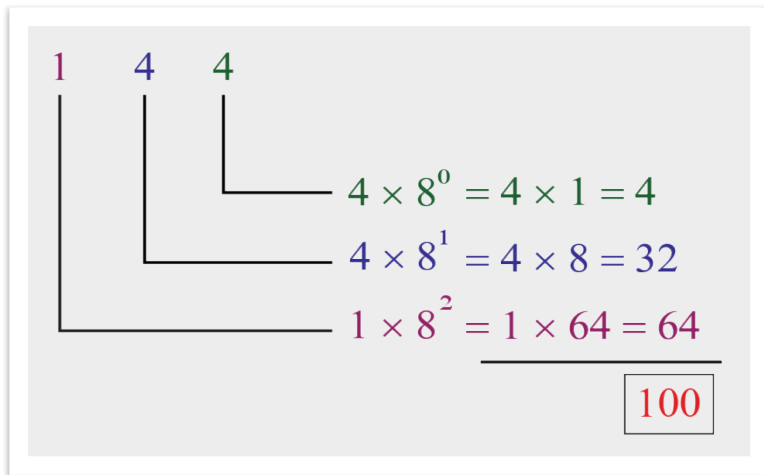
Solution: $5BC_{16}$ is in the hexadecimal system. We know that B = 11 and C = 12 in the hexadecimal system. So we get the equivalent number in the decimal system using the following process:



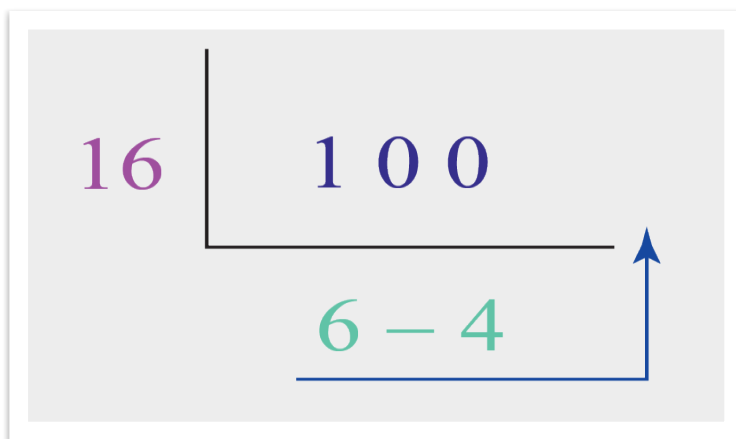
Thus, $5BC_{16} = 1468_{10}$

3. **Example 3:** Convert 144_8 into the hexadecimal system.

Solution: The base of 144_8 is 8. First, we will convert this number into the decimal system as follows:



Thus, $144_8 = 100_{10} \rightarrow (1)$. Now we will convert this into the hexadecimal system as follows:



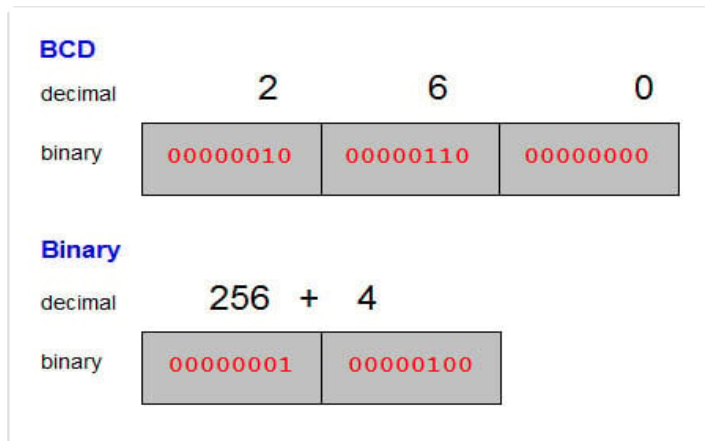
Thus, $100_{10} = 64_{16} \rightarrow (2)$

From the equations (1) and (2), we can conclude that: $144_8 = 64_{16}$

Binary Coded Decimal (BCD)

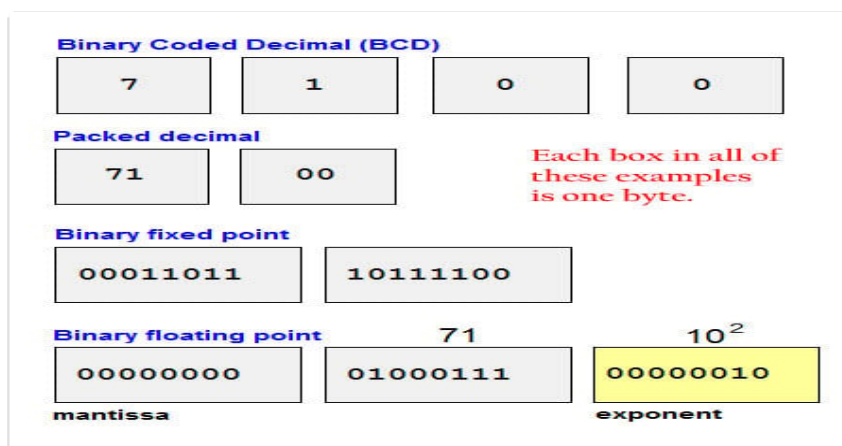
a number in binary code written in groups of four bits, each group representing one digit of the corresponding decimal number.

Binary Coded Decimal) The storage of numbers in the computer whereby each decimal digit is converted into a binary number and stored in a single 8-bit byte. For example, a 12-digit decimal number would be take 12 bytes. BCD uses more storage for numbers than binary encoding. Prior to 8-bit bytes in the 1960s, BCD encoding used 6-bit characters.



BCD and Binary The BCD method codes each decimal digit in binary and stores it in its own byte. The binary method converts the entire decimal number into a binary number. In the binary example above, the 1 in the left byte represents the value of 256 because it is the ninth bit from the right (256

- 128 64 32 16 8 4 2 1).



How Numbers Are Stored

BCD is one of four primary ways numbers are stored in the computer. **Binary Coded Decimal**, or **BCD**, is another process for converting decimal numbers into their binary equivalents.

- It is a form of binary encoding where each digit in a decimal number is represented in the form of bits.
- This encoding can be done in either 4-bit or 8-bit (usually 4-bit is preferred).
- It is a fast and efficient system that converts the decimal numbers into binary numbers as compared to the existing binary system.
- These are generally used in digital displays where the manipulation of data is quite a task.

- Thus, BCD plays an important role here because the manipulation is done treating each digit as a separate single sub-circuit.

The BCD equivalent of a decimal number is written by replacing each decimal digit in the integer and fractional parts with its four bit binary equivalent. The BCD code is more precisely known as 8421 BCD code, with 8,4,2 and 1 representing the weights of different bits in the four-bit groups, Starting from MSB and proceeding towards LSB. This feature makes it a weighted code, which means that each bit in the four bit group representing a given decimal digit has an assigned weight. Many decimal values, have an infinite place-value representation in binary but have a finite place-value in binary-coded decimal. For example, 0.2 in binary is .001100... and in BCD is 0.0010. It avoids fractional errors and is also used in huge financial calculations.

Consider the following truth table and focus on how are these represented.

Truth Table for Binary Coded Decimal

DECIMAL NUMBER	BCD
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

In the **BCD numbering system**, the given decimal number is segregated into chunks of four bits for each decimal digit within the number. Each decimal digit is converted into its direct binary form (usually represented in 4-bits).

For example:

1. Convert (123)₁₀ in BCD

From the truth table above,

1 -> 0001

2 -> 0010

3 -> 0011

thus, BCD becomes -> 0001 0010 0011

2. Convert (324)₁₀ in BCD

(324)₁₀ -> 0011 0010 0100 (BCD)

Again from the truth table above,

3 -> 0011

2 -> 0010

4 -> 0100

thus, BCD becomes -> 0011 0010 0100

This is how decimal numbers are converted to their equivalent BCDs.

- It is noticeable that the BCD is nothing more than a binary representation of each digit of a decimal number.
- It cannot be ignored that the BCD representation of the given decimal number uses extra bits, which makes it heavy-weighted.

.

ASCII and Parity

ASCII is used as a method to give all computers the same language, allowing them to share documents and files. ASCII is important because the development gave computers a common language.

ASCII an acronym for American Standard Code for Information Interchange, is a character encoding standard for electronic communication. ASCII codes represent text in computers, telecommunications equipment, and other devices.

The most widely accepted code is called the American Standard Code for Information Interchange (ASCII). The ASCII code associates an integer value for each symbol in the character set, such as letters, digits, punctuation marks, special characters, and control characters.

ASCII served its purpose for many decades, but Unicode has now effectively replaced it for all practical purposes other than legacy systems. Unicode is larger and, hence, more expressive. It represents a worldwide, collaborative effort and offers far greater flexibility, albeit at the expense of some complexity.

ASCII (American Standard Code for Information Interchange) is a standard character encoding used in telecommunication. The ASCII pronounced 'ask-ee', is strictly a seven-bit code based on the English alphabet. ASCII codes are used to represent alphanumeric data.

The code was first published as a standard in 1967. it was subsequently updated and published as ANSI X3.4-1968, then as ANSI X3.4-1977, and finally as ANSI X3.4-1986. Since it is a seven-bit code, it can at the most represent 128 characters. it currently defines 95 printable characters including 26 upper case letters (A to Z), 26 lower case letters, 10 numerals (0 to 9), and 33 special characters including mathematical symbols, punctuation marks, and space characters. They represent text in, telecommunications equipment, and devices. These include numbers, upper and lowercase English letters, functions, punctuation symbols, and some other symbols.

In total, there are 256 ASCII characters, and can be broadly divided into three categories:

1. ASCII control characters (0-31 and 127)
2. ASCII printable characters (32-126) (most commonly referred to)
3. Extended ASCII characters (128-255)

ASCII stands for American Standard Code for Information Interchange, A character encoding standard that assigns unique numerical values to letters, digits, punctuation marks, and other symbols.

In the physical world, you would use a pen and paper to write the message, and your friend would read the text directly. However, when it comes to digital communication, computers don't understand letters and symbols the way humans do. Instead, they process information in the form of binary code, which consists of 0s and 1s. This is where ASCII values come into play.

It can be observed that ASCII value of digits [0 – 9] ranges from [48 – 57]. Therefore, in order to print the ASCII value of any digit, 48 is required to be added to the digit.

In total, there are 256 ASCII characters, and can be broadly divided into three categories:

- ASCII control characters (0-31 and 127)
- ASCII printable characters (32-126) (most commonly referred to)
- Extended ASCII characters (128-255)

Parity

Parity is a mathematical operation that can be performed on binary numbers. It is a simple way of checking whether a number is even or odd. If the number of 1s in the binary representation of a number is even, then the number is even. If the number of 1s is odd, then the number is odd.

For a given set of bits, if the count of bits with a value of 1 is even, the parity bit value is set to 1 making the total count of 1s in the whole set (including the parity bit) an odd number. If the count of bits with a value of 1 is odd, the count is already odd so the parity bit's value is 0.

Unicode is an international character encoding standard that provides a unique number for every character across languages and scripts, making almost all characters accessible across platforms, programs, and devices.

ASCII is a 7-bit character set containing 128 characters. It contains the numbers from 0-9, the upper and lower case English letters from A to Z, and some special characters. The character sets used in modern computers, in HTML, and on the Internet, are all based on ASCII.

It enables developers to design interfaces that both humans and computers understand. ASCII codes a string of data as ASCII characters that can be interpreted and displayed as readable plain text for people and as data for computers. Programmers use the design of the ASCII character set to simplify certain tasks.

There are now two types of ASCII codes; the standard code that uses a seven-bit encoding system, and an extended code that uses an eight-bit system.

The ASCII table, an abbreviation for American Standard Code for Information Interchange, utilizes a 7-bit character encoding system to represent 128 unique characters. These include both control and printable characters.

The easiest method to convert binary to ASCII is to first convert binary to decimal. Then look up the ASCII character assigned to that decimal number. The ASCII lookup could also be scripted in your programming language of choice. You only count the decimal value if that position holds a binary value of 1

ASCII is a proper subset of Unicode, which means that Unicode includes all the characters that can be encoded in ASCII, plus many more. So, while ASCII is useful for representing characters in the English language, Unicode is capable of representing a much wider range of characters in other languages and scripts.

ASCII - Binary Character Table

Letter	ASCII Code	Binary	Letter	ASCII Code	Binary
a	097	01100001	A	065	01000001
b	098	01100010	B	066	01000010

c	099	01100011	C	067	01000011
d	100	01100100	D	068	01000100
e	101	01100101	E	069	01000101
f	102	01100110	F	070	01000110
g	103	01100111	G	071	01000111
h	104	01101000	H	072	01001000
i	105	01101001	I	073	01001001
j	106	01101010	J	074	01001010
k	107	01101011	K	075	01001011
l	108	01101100	L	076	01001100
m	109	01101101	M	077	01001101
n	110	01101110	N	078	01001110
o	111	01101111	O	079	01001111
p	112	01110000	P	080	01010000
q	113	01110001	Q	081	01010001
r	114	01110010	R	082	01010010
s	115	01110011	S	083	01010011
t	116	01110100	T	084	01010100
u	117	01110101	U	085	01010101
v	118	01110110	V	086	01010110
w	119	01110111	W	087	01010111
x	120	01111000	X	088	01011000
y	121	01111001	Y	089	01011001

z		122		01111010	Z		090		01011010
À	192	even	á	225	Odd				
Á	193	Odd	â	226	Even				
Â	194	Even	ã	227	Odd				
Ã	195	Odd	ä	228	Even				
Ä	196	Even	å	229	Odd				
Å	197	Odd	æ	230	Even				
Æ	198	Even	ç	231	Odd				
Ç	199	Odd	è	232	Even				
È	200	Even	é	233	Odd				
É	201	Odd	ê	234	Even				
Ê	202	Even	ë	235	Odd				
Ë	203	Odd	ì	236	Even				
Ì	204	Even	í	237	Odd				
Í	205	Odd	î	238	Even				
Î	206	Even	ï	239	Odd				
Ï	207	Odd	ð	240	Even				
Ñ	209	Odd	ñ	241	Odd				
Ò	210	Even	ò	242	Even				
Ó	211	Odd	ó	243	Odd				
Ô	212	Even	ô	244	Even				
Õ	213	Odd	õ	245	Odd				
Ö	214	Even	ö	246	Even				
Ø	216	Even	ø	248	Even				

Ù	217	Odd	ù	249	Odd
Ú	218	Even	ú	250	Even
Û	219	Odd	û	251	Odd
Ü	220	Even	ü	252	Even
à	224	Even	á	253	Odd
			ä	255	Odd

For example ASCII (American Standard Code for Information Interchange) is a character encoding standard used to represent text in computers and other devices that use text. ASCII codes are binary codes used to represent characters in a computer's memory. The ASCII code for the character 'X' is indeed 1011000.

Parity checking is a method used to check for errors in transmitted data. An even parity bit is added at the start of the data to make the total number of 1's in the data (including the parity bit) even.

Given that the 7-bit ASCII code for 'X' is 1011000, which has three 1's, an even parity bit would need to be added to make the total number of 1's even. Since the number of 1's is already odd, the parity bit should be 1 to make the total number of 1's even (four in this case).

So, the transmitted 8-bit code for the ASCII character 'X' with an even parity check bit would be 11011000.

You can print the ASCII values of the characters in the string "Hello world" using a simple C program. Here's a sample code snippet that demonstrates how to achieve this:

```

1. #include <stdio.h>
2.
3. int main() {
4.     char str[] = "Hello world";
5.
6.     printf("ASCII values of the characters in \"%s\":\n", str);
7.     for (int i = 0; str[i] != '\0'; i++) {
8.         printf("'c' = %d\n", str[i], str[i]);
9.     }
10.
11.     return 0;
12. }
```

Explanation:

1. Include the Standard I/O Library: The `#include <stdio.h>` directive includes the standard input-output library necessary for using the `printf` function.
2. Define the String: A character array `str` is defined and initialized with the string "Hello world".
3. Loop Through the String: A for loop iterates through each character in the string until it reaches the null terminator (`'\0'`).
4. Print ASCII Values: Inside the loop, `printf` is used to display each character along with its corresponding ASCII value, which is obtained by simply treating the character as an integer.

Output:

When you run this program, the output will display each character of "Hello world" along with its ASCII value, like this:

1. ASCII values of the characters in "Hello world":
2. 'H' = 72
3. 'e' = 101
4. 'l' = 108
5. 'l' = 108
6. 'o' = 111
7. ' ' = 32
8. 'w' = 119
9. 'o' = 111
10. 'r' = 114
11. 'l' = 108
12. 'd' = 100

You can compile and run this program using a C compiler to see the results

Boolean Algebra

In the 9th century, a Persian mathematician named Muhammad ibn Musa Al-Khwarizmi discovered a simpler way to dissect math problems. His contributions to the field of mathematics are now taught in classrooms every day.

Boolean algebra is a branch of mathematics that deals with operations on logical values with binary variables. Boolean variables are represented as binary numbers to represent truths: 1 = true and 0 = false. Elementary algebra deals with numerical operations, whereas Boolean algebra deals with logical operations.

In mathematics and mathematical logic, Boolean algebra is a branch of algebra. It differs from elementary algebra in two ways. First, the values of the variables are the truth values *true* and *false*, usually denoted 1 and 0, whereas in elementary algebra the values of the variables are numbers. Second, Boolean algebra uses logical operators such as conjunction (*and*) denoted as $\wedge$, disjunction (*or*) denoted as $\vee$, and negation (*not*) denoted as $\neg$.

Elementary algebra, on the other hand, uses arithmetic operators such as addition, multiplication, subtraction, and division. Boolean algebra is therefore a formal way of describing logical operations in the same way that elementary algebra describes numerical operations.

Boolean Algebra is fundamental in the development of digital electronics systems as they all use the concept of Boolean Algebra to execute commands. Apart from digital electronics this algebra also finds its application in Set Theory, Statistics, and other branches of mathematics.

Boolean algebra was introduced by George Boole in his first book *The Mathematical Analysis of Logic* (1847)

Boolean is a data type that represents a logical value, which can be either true or false.

Named after the mathematician George Boole, who developed Boolean algebra, Boolean values are fundamental in programming for making decisions.

Boolean algebra is the category of algebra in which the variable's values are the truth values, **true and false**, ordinarily denoted 1 and 0 respectively. It is used to analyse and simplify digital circuits or digital gates. It is also called Binary Algebra or logical Algebra. It has been fundamental in the development of digital electronics and is provided for in all modern programming languages. It is also used in set theory and statistics.

The algebra of logic is a Boolean algebra. It works with variables with two different values, such as 0 (False) and 1 (True), as well as logically significant operations. George Boole invented the first way of manipulating symbolic logic, which later became known as Boolean Algebra. Because of its vast applications in switching theory, developing basic electronic

circuits, and designing digital computers, Boolean algebra has become a vital tool in computer science.

The following are the basic operations of Boolean algebra:

- AND or conjunction operation
- OR operation or disjunction
- Not or Negative operation

Below is the table defining the symbols for all three basic operations.

Operator	Symbol	Precedence
NOT	' (or) $\neg$	Highest
AND	. (or) $\wedge$	Middle
OR	+ (or) $\vee$	Lowest

The important operations performed in Boolean algebra are – **conjunction ($\wedge$)**, **disjunction ($\vee$)** and **negation ($\neg$)**. Hence, this algebra is far way different from elementary algebra where the values of variables are numerical and arithmetic operations like addition, subtraction is been performed on them.

There are three basic operations in Boolean algebra: conjunction, disjunction, and negation. These Boolean operations are expressed with the corresponding logical operators AND, OR, and NOT, collectively referred to as Boolean operators

Boolean Expression

A logical statement that results in a Boolean value, either be True or False, is a Boolean expression. Sometimes, synonyms are used to express the statement such as 'Yes' for 'True' and 'No' for 'False'. Also, 1 and 0 are used for digital circuits for True and False, respectively.

Boolean expressions are the statements that use logical operators, i.e., AND, OR, XOR and NOT. Thus, if we write $X \text{ AND } Y = \text{True}$, then it is a Boolean expression.

Rules:

Suppose X and Y are two Boolean variables, then the three operations are defined as follows;

- $X \text{ AND } Y$, satisfies $X \wedge Y = \text{True}$, if $X = Y = \text{True}$ or else $X \wedge Y = \text{False}$.

- $X \text{ OR } Y$, satisfies $X \vee Y = \text{False}$, if $X = Y = \text{False}$, else $X \vee Y = \text{True}$.
- $\neg X$ satisfies $\neg X = \text{False}$, if $X = \text{True}$ and $\neg X = \text{True}$ if $X = \text{False}$

Boolean Algebra Terminologies

Now, let us discuss the important terminologies covered in Boolean algebra.

Boolean Algebra: Boolean algebra is the branch of algebra that deals with logical operations and binary variables.

Boolean Variables: A Boolean variable is defined as a variable or a symbol defined as a variable or a symbol, generally an alphabet that represents the logical quantities such as 0 or 1.

Boolean Function: A Boolean function consists of binary variables, logical operators, constants such as 0 and 1, equal to the operator, and the parenthesis symbols.

Literal: A literal may be a variable or a complement of a variable.

Complement: The complement is defined as the inverse of a variable, which is represented by a bar over the variable.

Truth Table: The truth table is a table that gives all the possible values of logical variables and the combination of the variables. It is possible to convert the Boolean equation into a truth table. The number of rows in the truth table should be equal to 2^n , where “n” is the number of variables in the equation. For example, if a Boolean equation consists of 3 variables, then the number of rows in the truth table is 8. (i.e.,) $2^3 = 8$.

A truth table provides a method for mapping out the possible truth values in an expression and to determine their outcomes. The table includes a column for each variable in the expression and a row for each possible combination of truth values. It also includes a column that shows the outcome of each set of values.

Boolean Algebra Truth Table

Now, if we express the above operations in a truth table, we get;

A	B	$A \wedge B$	$A \vee B$
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

A	$\neg A$
True	False
False	True

Boolean Algebra Rules

Following are the important rules used in Boolean algebra.

- Variable used can have only two values. Binary 1 for HIGH and Binary 0 for LOW.
- The complement of a variable is represented by an overbar.
- OR-ing of the variables is represented by a plus (+) sign between them. For example, the OR-ing of A, B, and C is represented as $A + B + C$.
- Logical AND-ing of the two or more variables is represented by writing a dot between them, such as $A.B.C$. Sometimes, the dot may be omitted like ABC .

Laws of Boolean Algebra

There are six types of Boolean algebra laws. They are:

1. Commutative Law

Any binary operation which satisfies the following expression is referred to as a commutative operation. Commutative law states that changing the sequence of the variables does not have any effect on the output of a logic circuit.

- $A.B = B.A$
- $A + B = B + A$

2. Associative Law

It states that the order in which the logic operations are performed is irrelevant as their effect is the same.

- $(A.B).C = A.(B.C)$
- $(A + B) + C = A + (B + C)$

3. Distributive Law

Distributive law states the following conditions:

- $A.(B + C) = (A.B) + (A.C)$
- $A + (B.C) = (A + B).(A + C)$

AND Law

These laws use the AND operation. Therefore they are called AND laws.

- $A \cdot 0 = 0$
- $A \cdot 1 = A$
- $A \cdot A = A$
-

OR Law

These laws use the OR operation. Therefore they are called OR laws.

- $A + 0 = A$
- $A + 1 = 1$
- $A + A = A$
-

4. Inversion Law

In Boolean algebra, the inversion law states that double inversion of variable results in the original variable itself.

-

Boolean Algebra Theorems

The two important theorems which are extremely used in Boolean algebra are De Morgan's First law and De Morgan's second law. These two theorems are used to change the Boolean expression. This theorem basically helps to reduce the given Boolean expression in the simplified form. These two De Morgan's laws are used to change the expression from one form to another form. Now, let us discuss these two theorems in detail.

5. De Morgan's First Law:

De Morgan's First Law states that $(A \cdot B)' = A' + B'$.

The first law states that the complement of the product of the variables is equal to the sum of their individual complements of a variable.

The truth table that shows the verification of De Morgan's First law is given as follows:

A	B	A'	B'	(A.B)'	A'+B'
0	0	1	1	1	1

0	1	1	0	1	1
1	0	0	1	1	1
1	1	0	0	0	0

The last two columns show that $(A.B)' = A' + B'$.

Hence, De Morgan's First Law is proved.

6. De Morgan's Second Law:

De Morgan's Second law states that $(A+B)' = A' . B'$.

The second law states that the complement of the sum of variables is equal to the product of their individual complements of a variable.

The following truth table shows the proof for De Morgan's second law.

A	B	A'	B'	$(A+B)'$	$A' . B'$
0	0	1	1	1	1
0	1	1	0	0	0
1	0	0	1	0	0
1	1	0	0	0	0

The last two columns show that $(A+B)' = A' . B'$.

Hence, De Morgan's second law is proved.

The other theorems in Boolean algebra are complementary theorem, duality theorem, transposition theorem, redundancy theorem and so on. All these theorems are used to simplify the given Boolean expression. The reduced Boolean expression should be equivalent to the given Boolean expression.

Solved Examples

Question: Draw a truth table for $A(B+D)$.

Solution: Given expression $A(B+D)$.

A	B	D	$B+D$	$A(B+D)$
0	0	0	0	0
0	0	1	1	0

0	1	0	1	0
0	1	1	1	0
1	0	0	0	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1